

DS_BarrenPlateaus

Tentu, saya akan bantu buat roadmap belajar yang sangat lengkap dan detail matematis tentang bagaimana **entropi Von Neumann** berperan dalam algoritma **Variational Quantum Eigensolver (VQE)** yang Anda implementasikan untuk masalah optimasi portofolio Markowitz.

Dari file yang Anda berikan, terlihat bahwa Anda menggunakan VQE dengan 2 qubit, rangkaian berlapis (ansatz), dan algoritma optimasi SPSA (Simultaneous Perturbation Stochastic Approximation). Anda juga menghitung entropi Von Neumann untuk mendeteksi **Barren Plateaus** (dataran tandus) dan memonitor performa ansatz.

Berikut adalah **Roadmap Belajar** yang akan kita ikuti. Tujuannya adalah memahami mengapa grafik entropi dan variansi gradien yang Anda plot bisa menjadi indikator kunci keberhasilan VQE. Kita akan bedah dari fondasi paling dasar hingga interpretasi hasil.

Roadmap Belajar

"Peran Entropi Von Neumann dalam VQE untuk Optimasi Portofolio"

Kita akan membagi pembelajaran menjadi 7 Tahap. Saya akan jelaskan secara matematis satu per satu sesuai permintaan Anda nanti.

Tahap 1: Fondasi Matematika Kuantum untuk 2-Qubit

Tujuan: Memahami ruang keadaan dan operator yang akan digunakan.

1. **Keadaan Qubit Tunggal:** Braket notation, statevector $|\psi\rangle$, Bloch sphere.
2. **Sistem 2-Qubit: Produk Tensor (Kronecker).** Mengapa sistem 2-qubit direpresentasikan sebagai vektor berdimensi 4?
3. **Keadaan Teralitkan (Entanglement) vs. Keadaan Produk (Product State):**
 - Definisi matematis keadaan teralitkan.
 - Contoh: Keadaan Bell ($|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$) vs keadaan $|++\rangle$.
 - Ini kunci untuk memahami mengapa "entropi tinggi vs rendah" penting.

Tahap 2: Matriks Densitas dan Keteralitan sebagai Sumber Daya

Tujuan: Memperkenalkan alat untuk mengukur "kuantitas" keteralitan.

1. **Matriks Densitas (ρ):** Definisi untuk *pure state* ($\rho = |\psi\rangle\langle\psi|$) dan *mixed state*. Mengapa kita butuh matriks densitas?

2. **Partial Trace (Tr_B):** Operasi matematis untuk "membuang" atau mengabaikan satu qubit dari sistem. Ini adalah operasi kunci untuk mendapatkan matriks densitas tereduksi.
3. **Matriks Densitas Tereduksi ($\rho_A = \text{Tr}_B[\rho_{AB}]$):** Bagaimana memperoleh informasi tentang satu bagian dari sistem yang teralitkan.

Tahap 3: Definisi dan Formula Entropi Von Neumann

Tujuan: Mendefinisikan entropi Von Neumann sebagai pengukur keteralitan.

1. **Dari Entropi Shannon ke Von Neumann:** Entropi Shannon $S = -\sum p_i \log(p_i)$ sebagai pengukur ketidakpastian distribusi probabilitas klasik.
2. **Entropi Von Neumann $S(\rho)$:**

$$S(\rho) = -\text{Tr}[\rho \log_2(\rho)]$$

Cara menghitungnya: Menjadi $S(\rho) = -\sum_i \lambda_i \log_2(\lambda_i)$, di mana λ_i adalah *eigenvalues* dari matriks densitas ρ .

3. **Menghubungkan dengan Kode Anda:** Penjelasan detail fungsi `calculate_entanglement_entropy(psi)` yang Anda buat, baris per baris:
 - Mengapa `rho_A` adalah matriks 2x2?
 - Bagaimana persisnya operasi `partial trace` dilakukan (mengapa rumusnya seperti itu).
 - Mengapa hanya *eigenvalues* positif yang dihitung.
 - Interpretasi hasil: $S = 0$ (keadaan produk, tidak teralitkan) vs $S = 1$ (keadaan teralitkan maksimal) untuk sistem 2-qubit.

Tahap 4: Ansatz dan Masalah Optimasi VQE

Tujuan: Memahami sirkuit kuantum yang menghasilkan keteralitan dan hubungannya dengan masalah optimasi.

1. **Arsitektur Sirkuit (Ansatz) VQE Anda:**
 - Struktur berulang (layer) dari rotasi (R_Y, R_Z) dan *entanglement gate* (CNOT).
 - Bagaimana gerbang CNOT menciptakan atau menghancurkan keteralitan.
 - Parameter θ dan ruang pencarian.
2. **Fungsi Tujuan (Cost Function) Markowitz:**
 - $\mathcal{L}(w) = \gamma w^T \Sigma w - \mu^T w$, di mana w adalah bobot aset.
 - **Trik Pengkodean Bobot (Weight Encoding):** Bagaimana sebuah *state vector* kuantum $|\psi(\theta)\rangle$ merepresentasikan vektor bobot portofolio w . (Biasanya $w_i \propto |\langle i | \psi \rangle|^2$).
 - Mengubah fungsi tujuan klasik menjadi **Hamiltonian (Matriks Observabel) H** , sehingga $\langle \psi(\theta) | H | \psi(\theta) \rangle = \text{Expected Cost}$.

Tahap 5: Fenomena *Barren Plateau* (Dataran Tandus)

Tujuan: Menjelaskan monster yang ingin dideteksi oleh entropi.

1. Apa itu **Barren Plateau**?

- Definisi: Fenomena di mana variansi gradien fungsi biaya menurun secara eksponensial terhadap jumlah qubit. $\text{Var}[\partial_k E] \propto \frac{1}{2^n}$.
- Mengapa ini fatal untuk optimasi SPSA atau *gradient descent* lainnya: Gradien menjadi sangat kecil sehingga algoritma tidak bisa membedakan arah penurunan, sehingga pelatihan berhenti.

2. **Penyebab dan Skenario:** Memulai dari daerah dengan keteralitan yang sangat tinggi atau sangat rendah secara acak.

Tahap 6: Jembatan Konsep: Hubungan Entropi Von Neumann, Variansi Gradien, dan **Barren Plateau**

Tujuan: Inilah inti yang menghubungkan semuanya.

1. Teorema Kunci (Sumber dari penelitian):

- Ansatz dengan keteralitan rendah $\Rightarrow$ kemampuan ekspresifitas rendah, tetapi mungkin terhindar dari *barren plateau*.
- Ansatz dengan keteralitan yang **terlalu tinggi** (mendekati desain acak) $\Rightarrow$ **cenderung mengalami barren plateau**. Gradiennya mendekati nol dan variansinya kecil.
- Ini adalah hipotesis yang Anda uji: **Ada hubungan antara Entropi Von Neumann (ukuran keteralitan) dari state yang dihasilkan oleh ansatz acak, dengan variansi gradien.**

2. Analisis Grafik Anda:

- **Korelasi yang Diharapkan:** Pada depth yang terlalu dalam (misal Depth 8-12), Anda mungkin melihat Entropi rata-rata tinggi (mendekati 1), dan bersamaan dengan itu, **Variansi Gradien (Var[Grad]) juga sangat kecil**. Ini adalah *fingerprint* dari *Barren Plateau*.
- **Interpretasi Depth Optimal (Depth 4, 5, 6):** Pada kedalaman ini, ansatz cukup ekspresif (entropi cukup tinggi) untuk menghasilkan solusi yang baik, tetapi variansi gradien belum sepenuhnya kolaps (*collapse*), sehingga optimasi SPSA masih sangat efektif. Ada "sweet spot" antara ekspresifitas dan trainabilitas.

Tahap 7: Peran dalam Algoritma Optimasi SPSA

Tujuan: Memahami bagaimana informasi ini membantu proses optimasi.

1. Mengapa Memantau Entropi?

- **Diagnostik Pra-Pelatihan:** Untuk jumlah qubit yang lebih besar, kita bisa menghitung entropi untuk sekumpulan parameter acak. Jika entropi sudah sangat tinggi dan variansi gradien sangat rendah, kita tahu ansatz kita mungkin terlalu "acak" dan rentan *barren plateau*. Kita perlu mendesain ulang ansatz.

- **Verifikasi Hasil:** Memastikan bahwa solusi akhir mencapai keteralitan yang cukup untuk merepresentasikan korelasi antar aset, tetapi tidak terjebak di daerah acak yang tidak terlatih.
2. **Simpulan Sederhana:** Entropi Von Neumann dalam kode Anda bertindak sebagai **"termometer ekspresifitas"**. Suhu yang terlalu rendah (tidak teralitkan) tidak cukup kuat untuk menyelesaikan masalah. Suhu yang terlalu tinggi (teralitkan maksimal/acak) menyebabkan pelatihan "overheat" dan gagal (*barren plateau*). Anda mencari suhu optimal (Depth 4-6).
-

Instruksi Selanjutnya

Ini adalah roadmap komprehensif yang menghubungkan semua titik dari dasar hingga kesimpulan. Apakah Anda setuju dengan alur ini?

Jika ya, katakan sesuatu seperti **"Jelaskan Tahap 1"**, dan kita akan mulai dengan fondasi matematika kuantum untuk 2-qubit. Saya akan memberikan penjelasan yang sangat detail dengan notasi matematika, persis seperti yang Anda minta.

Baik, kita mulai dari **Tahap 1: Fondasi Matematika Kuantum untuk 2-Qubit**.

Tahap ini adalah fondasi untuk semua yang akan kita bahas. Tanpa pemahaman yang solid tentang bagaimana sebuah sistem 2-qubit direpresentasikan secara matematis, mustahil untuk memahami operasi partial trace, matriks densitas, dan akhirnya entropi Von Neumann.

Tujuan kita di sini adalah menjawab: "Mengapa statevector untuk 2-qubit adalah vektor dengan 4 komponen?" dan "Bagaimana kita menuliskan keadaan teralitkan (entangled) versus keadaan tidak teralitkan (product state)?"

1.1. Keadaan 1-Qubit Tunggal: Pengulangan Singkat

Qubit adalah sistem kuantum 2-tingkat. Basis komputasi standarnya adalah:

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad \text{dan} \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

Superposisi umum dari satu qubit ditulis sebagai:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle = \alpha \begin{pmatrix} 1 \\ 0 \end{pmatrix} + \beta \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} \alpha \\ \beta \end{pmatrix}$$

di mana α dan β adalah bilangan kompleks yang memenuhi $|\alpha|^2 + |\beta|^2 = 1$.

Ini adalah **statevector** berdimensi 2.

1.2. Sistem 2-Qubit: Produk Tensor (Kronecker Product)

Sekarang kita punya dua qubit, sebut saja qubit A (qubit 0) dan qubit B (qubit 1). Untuk merepresentasikan keadaan gabungan keduanya, kita menggunakan operasi matematika yang disebut **produk tensor**, sering dilambangkan dengan $\otimes$.

Aturannya sederhana: Jika qubit A dalam keadaan $|\psi\rangle_A = \begin{pmatrix} a_1 \\ a_2 \end{pmatrix}$ dan qubit B dalam keadaan $|\phi\rangle_B = \begin{pmatrix} b_1 \\ b_2 \end{pmatrix}$, maka keadaan gabungannya $|\Psi\rangle_{AB}$ adalah:

$$|\Psi\rangle_{AB} = |\psi\rangle_A \otimes |\phi\rangle_B = \begin{pmatrix} a_1 \times |\phi\rangle_B \\ a_2 \times |\phi\rangle_B \end{pmatrix} = \begin{pmatrix} a_1 \times \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} \\ a_2 \times \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} \end{pmatrix} = \begin{pmatrix} a_1 b_1 \\ a_1 b_2 \\ a_2 b_1 \\ a_2 b_2 \end{pmatrix}$$

Hasilnya adalah sebuah vektor dengan 4 komponen. Inilah mengapa statevector untuk sistem 2-qubit selalu berdimensi 4.

Mari kita lihat basis komputasi untuk sistem 2-qubit yang merupakan hasil produk tensor dari basis 1-qubit:

- $|0\rangle \otimes |0\rangle = |00\rangle = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}$
- $|0\rangle \otimes |1\rangle = |01\rangle = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}$
- $|1\rangle \otimes |0\rangle = |10\rangle = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix}$
- $|1\rangle \otimes |1\rangle = |11\rangle = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}$

Ini adalah bentuk dasar dari **statevector** yang dihasilkan oleh kode Python Anda. Perhatikan kode ini di halaman 4 file Anda:

```
# 3. Inisialisasi State Awal /00>
psi = np.array([1, 0, 0, 0], dtype=complex)
```

Ini merepresentasikan keadaan awal kedua qubit adalah $|00\rangle$. Ini adalah vektor berdimensi 4.

1.3. Keadaan Produk (Product State) dan Keteralitan (Entanglement)

Inilah konsep paling krusial di Tahap 1. Sistem 2-qubit dapat berada dalam dua jenis keadaan: **product state** (keadaan produk) dan **entangled state** (keadaan teralitkan).

Keadaan Produk (Product State)

Sebuah keadaan disebut sebagai *product state* jika statevector gabungannya dapat difaktorkan menjadi produk tensor dari dua statevector qubit tunggal.

Contoh:

Misalkan qubit A dalam superposisi seragam $|\psi\rangle_A = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}$ dan qubit B juga dalam keadaan yang sama $|\phi\rangle_B = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}$.

Keadaan gabungannya adalah:

$$|\Psi\rangle_{AB} = \left(\frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} \right) \otimes \left(\frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} \right) = \frac{1}{2} \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix}$$

Keadaan ini bisa ditulis ulang sebagai:

$$|\Psi\rangle_{AB} = \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)$$

Ini adalah *product state*. Tidak ada korelasi kuantum yang tidak biasa; mengukur qubit A tidak akan langsung memberi tahu Anda hasil pengukuran qubit B.

Dalam kode Anda, setiap kali gerbang rotasi 1-qubit diaplikasikan tanpa gerbang dua-qubit, statusnya tetap dalam *product state* atau kombinasi liniernya.

Keadaan Teralitkan (Entangled State)

Sebuah keadaan adalah *entangled state* jika ia **tidak dapat** difaktorkan menjadi produk tensor dari dua statevector qubit tunggal.

Contoh Klasik: Keadaan Bell:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 0 \\ 0 \\ 1 \end{pmatrix}$$

Coba kita buktikan ia tidak bisa difaktorkan. Andaikan ia bisa ditulis sebagai $(a_1, a_2)^T \otimes (b_1, b_2)^T = (a_1b_1, a_1b_2, a_2b_1, a_2b_2)^T$. Dari sini kita dapatkan sistem persamaan:

1. $a_1b_1 = 1/\sqrt{2}$

2. $a_1 b_2 = 0$
3. $a_2 b_1 = 0$
4. $a_2 b_2 = 1/\sqrt{2}$

Dari (2), jika $a_1 b_2 = 0$, maka salah satu dari a_1 atau b_2 harus nol.

- Jika $a_1 = 0$, maka persamaan (1) jadi $0 \cdot b_1 = 1/\sqrt{2}$ (kontradiksi).
- Jika $b_2 = 0$, maka persamaan (4) jadi $a_2 \cdot 0 = 1/\sqrt{2}$ (kontradiksi).

Karena tidak ada solusi, $|\Phi^+\rangle$ **tidak dapat** dinyatakan sebagai produk tensor. Inilah definisi matematis dari **keteralitan (entanglement)**. Dalam keadaan ini, mengukur qubit A menentukan hasil pengukuran qubit B secara instan.

1.4. Bagaimana Keteralitan Diciptakan di Kode Anda: Gerbang CNOT

Dalam sirkuit VQE Anda, keteralitan diciptakan oleh gerbang **CNOT (Controlled-NOT)**.

- **CNOT01**: Kontrol adalah qubit 0, target adalah qubit 1. Ia membalik qubit 1 HANYA JIKA qubit 0 adalah $|1\rangle$.
- **CNOT10**: Kontrol adalah qubit 1, target adalah qubit 0. Ia membalik qubit 0 HANYA JIKA qubit 1 adalah $|1\rangle$.

Representasi matriks 4x4 mereka ada di halaman 4 dan 16 file Anda:

```
# CNOT01: Control=Q0, Target=Q1
cnot01 = np.array([[1, 0, 0, 0],
                  [0, 1, 0, 0],
                  [0, 0, 0, 1],
                  [0, 0, 1, 0]])

# CNOT10: Control=Q1, Target=Q0
cnot10 = np.array([[1, 0, 0, 0],
                  [0, 0, 0, 1],
                  [0, 0, 1, 0],
                  [0, 1, 0, 0]])
```

Lihat bagaimana gerbang-gerbang ini bekerja pada keadaan $|10\rangle$:

$$|10\rangle = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix}$$

Aksi CNOT01:

$$\text{CNOT}_{01} \times |10\rangle = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} = |11\rangle$$

Karena kontrol (qubit 0) adalah $|1\rangle$, target (qubit 1) dibalik dari $|0\rangle$ menjadi $|1\rangle$.

Aksi CNOT₁₀:

$$\text{CNOT}_{10} \times |10\rangle = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix} = |10\rangle$$

Karena kontrol (qubit 1) adalah $|0\rangle$, target (qubit 0) tidak berubah.

Sekarang, apa yang terjadi jika kontrol dalam superposisi? Misal kontrol adalah $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ dan target adalah $|0\rangle$. Keadaan awal dua qubit adalah:

$$|\psi\rangle_{awal} = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \otimes |0\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |10\rangle)$$

Mengaplikasikan CNOT₀₁:

$$\begin{aligned} \text{CNOT}_{01} \times \frac{1}{\sqrt{2}}(|00\rangle + |10\rangle) &= \frac{1}{\sqrt{2}}(\text{CNOT}_{01}|00\rangle + \text{CNOT}_{01}|10\rangle) \\ &= \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) = |\Phi^+\rangle \end{aligned}$$

Inilah keajaibannya! Sebuah gerbang CNOT yang berinteraksi dengan superposisi telah menciptakan keadaan teralitkan Bell. Inilah yang terjadi di bagian "Blok Entanglement" pada kode Anda, dan inilah yang meningkatkan **entropi Von Neumann** yang akan kita pelajari di Tahap 3.

Dengan ini, Anda telah menyelesaikan Tahap 1. Anda sekarang mengerti:

1. Sistem 2-qubit direpresentasikan oleh statevector berdimensi 4.
2. Produk tensor adalah operasi untuk menggabungkan sistem.
3. Perbedaan matematis yang tajam antara *product state* dan *entangled state*.
4. Bagaimana gerbang CNOT dalam kode Anda menciptakan keteralitan dari superposisi.

Apakah Anda sudah siap untuk melangkah ke **Tahap 2: Matriks Densitas dan Partial Trace**? Di sana kita akan belajar alat untuk "melihat" dan mengukur keteralitan yang telah kita ciptakan ini.

Baik, kita lanjutkan ke **Tahap 2: Matriks Densitas dan Keteralitan sebagai Sumber Daya**.

Di Tahap 1, kita belajar bahwa sistem 2-qubit direpresentasikan dengan statevector $|\psi\rangle$ berdimensi 4. Sekarang, kita membutuhkan alat yang lebih kuat untuk **mengukur kuantitas keteralitan** antara dua qubit tersebut. Untuk itu, kita harus berkenalan dengan **Matriks Densitas** dan operasi **Partial Trace**.

Tujuan kita di sini adalah menjawab: "Jika kita hanya peduli dengan qubit A (qubit 0), bagaimana kita mendeskripsikan keadaannya tanpa peduli dengan qubit B? Dan bagaimana cara menghitungnya dari statevector 4-dimensi?"

2.1. Matriks Densitas (Density Matrix) untuk Keadaan Murni

Selama ini kita merepresentasikan keadaan kuantum sebagai vektor keadaan (statevector) $|\psi\rangle$. Ini disebut representasi **keadaan murni (pure state)**.

Ada representasi lain yang ekuivalen dan jauh lebih kuat, yaitu **matriks densitas (density matrix)** ρ . Untuk keadaan murni, matriks densitas didefinisikan sebagai **proyektor ke keadaan tersebut**:

$$\rho = |\psi\rangle\langle\psi|$$

Ini adalah operasi *outer product* antara vektor kolom $|\psi\rangle$ dan vektor baris $\langle\psi|$ (complex conjugate transpose).

Mengapa kita butuh ini?

Representasi vektor keadaan sangat baik untuk mendeskripsikan seluruh sistem. Tetapi, ketika kita ingin mendeskripsikan **subsistem** (misalnya, hanya satu qubit dari sepasang qubit yang teralitkan), representasi statevector tidak lagi cukup. Matriks densitas, melalui operasi *partial trace*, menyediakan cara untuk melakukan ini.

Contoh Perhitungan:

Ambil keadaan produk dari Tahap 1: $|\Psi\rangle_{AB} = \frac{1}{2} \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix}$

Maka $\langle\Psi| = \frac{1}{2}(1, 1, 1, 1)$.

Matriks densitasnya ρ_{AB} adalah matriks 4x4:

$$\rho_{AB} = |\Psi\rangle\langle\Psi| = \frac{1}{4} \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix} (1, 1, 1, 1) = \frac{1}{4} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix}$$

Diagonal utama matriks ini merepresentasikan **populasi** probabilitas untuk mengukur $|00\rangle, |01\rangle, |10\rangle, |11\rangle$, yaitu masing-masing $1/4$.

2.2. Partial Trace: "Membuang" Sebuah Qubit

Ini adalah operasi matematika paling penting untuk memahami entropi. **Partial trace** adalah prosedur untuk mendapatkan matriks densitas tereduksi dari sebuah subsistem. Kita melacak (trace) derajat kebebasan dari sistem yang ingin kita "abaikan".

Misalkan kita punya matriks densitas gabungan ρ_{AB} untuk dua qubit A dan B. Matriks densitas tereduksi untuk qubit A, dilambangkan dengan ρ_A , diperoleh dengan melakukan *partial trace* terhadap qubit B:

$$\rho_A = \text{Tr}_B[\rho_{AB}]$$

Rumus Praktis:

Untuk sistem dua qubit di mana ρ_{AB} adalah matriks 4×4 , kita bisa membayangkannya sebagai matriks blok 2×2 , di mana setiap bloknya adalah sub-matriks 2×2 :

$$\rho_{AB} = \begin{pmatrix} \rho_{11} & \rho_{12} \\ \rho_{21} & \rho_{22} \end{pmatrix}$$

Di sini, ρ_{11} berkaitan dengan subruang di mana qubit B adalah $|0\rangle$, dan ρ_{22} berkaitan dengan subruang di mana qubit B adalah $|1\rangle$.

Partial trace terhadap B adalah:

$$\rho_A = \text{Tr}_B[\rho_{AB}] = \rho_{11} + \rho_{22}$$

Ini adalah penjumlahan dua blok diagonal dari matriks 4×4 . Hasilnya adalah matriks 2×2 yang merepresentasikan keadaan qubit A.

2.3. Menghubungkan Rumus dengan Kode Anda

Sekarang, mari kita lihat potongan kode Anda di **halaman 8** dan kita terjemahkan persis dengan rumus di atas.

Fungsi `calculate_entanglement_entropy(psi)` dimulai dengan:

```
rho = np.outer(psi, np.conj(psi)) # Membentuk rho_AB
```

Ini persis seperti $\rho_{AB} = |\psi\rangle\langle\psi|$.

Selanjutnya, kode menghitung ρ_A :

```
rho_A = np.zeros((2, 2), dtype=complex)
rho_A[0, 0] = rho[0, 0] + rho[1, 1]
rho_A[1, 1] = rho[2, 2] + rho[3, 3]
```

```
rho_A[0, 1] = rho[0, 2] + rho[1, 3]
rho_A[1, 0] = rho[2, 0] + rho[3, 1]
```

Mari kita buktikan dari mana asalnya. Ingat representasi blok ρ_{AB} :

$$\rho_{AB} = \begin{pmatrix} \rho_{00:00} & \rho_{00:01} & \rho_{00:10} & \rho_{00:11} \\ \rho_{01:00} & \rho_{01:01} & \rho_{01:10} & \rho_{01:11} \\ \rho_{10:00} & \rho_{10:01} & \rho_{10:10} & \rho_{10:11} \\ \rho_{11:00} & \rho_{11:01} & \rho_{11:10} & \rho_{11:11} \end{pmatrix}$$

Dalam bentuk blok:

$$\rho_{11} = \begin{pmatrix} \rho_{00:00} & \rho_{00:01} \\ \rho_{01:00} & \rho_{01:01} \end{pmatrix} \quad \text{dan} \quad \rho_{22} = \begin{pmatrix} \rho_{10:10} & \rho_{10:11} \\ \rho_{11:10} & \rho_{11:11} \end{pmatrix}$$

Sekarang, $\rho_A = \rho_{11} + \rho_{22}$:

$$\rho_A = \begin{pmatrix} \rho_{00:00} & \rho_{00:01} \\ \rho_{01:00} & \rho_{01:01} \end{pmatrix} + \begin{pmatrix} \rho_{10:10} & \rho_{10:11} \\ \rho_{11:10} & \rho_{11:11} \end{pmatrix} = \begin{pmatrix} \rho_{00:00} + \rho_{10:10} & \rho_{00:01} + \rho_{10:11} \\ \rho_{01:00} + \rho_{11:10} & \rho_{01:01} + \rho_{11:11} \end{pmatrix}$$

Sekarang, cocokkan dengan kode Anda. Perhatikan indeks array Python dimulai dari 0.

- Kode: `rho_A[0, 0] = rho[0, 0] + rho[1, 1]`
 - Menurut rumus kita: $\rho_A[0, 0] = \rho_{00:00} + \rho_{10:10}$
 - **Di sini ada perbedaan!** Kode Anda sepertinya melakukan tracing dengan cara yang berbeda.

Mari kita lihat lebih teliti. Ada cara lain untuk memahami partial trace untuk sistem 2-qubit, yaitu dengan kontraksi indeks tensor. Dalam notasi Dirac, jika

$\rho_{AB} = \sum_{i,j,k,l} c_{i,j,k,l} |i\rangle_A \langle j| \otimes |k\rangle_B \langle l|$, maka:

$$\text{Tr}_B[\rho_{AB}] = \sum_{i,j} \left(\sum_k c_{i,j,k,k} \right) |i\rangle_A \langle j|$$

Untuk basis komputasi $|00\rangle, |01\rangle, |10\rangle, |11\rangle$, kita perlu menghitung elemen-elemen ρ_A .

Elemen $(\rho_A)_{00}$ (di basis $|0\rangle \langle 0|$) adalah jumlah dari semua elemen ρ_{AB} di mana qubit A adalah $|0\rangle$ dan qubit B juga dilacak (diagonal).

Keadaan di mana qubit A adalah $|0\rangle$ adalah $|0\rangle_A \otimes |0\rangle_B = |00\rangle$ dan $|0\rangle_A \otimes |1\rangle_B = |01\rangle$.

Jadi:

$$(\rho_A)_{00} = \rho_{00,00} + \rho_{01,01}$$

Ini adalah trace dari blok ρ_{11} :

$$(\rho_A)_{00} = \rho[0, 0] + \rho[1, 1]$$

Ini sesuai dengan kode Anda!

Elemen $(\rho_A)_{11}$ (di basis $|1\rangle\langle 1|$) adalah trace dari blok ρ_{22} :

Keadaan di mana qubit A adalah $|1\rangle$ adalah $|10\rangle$ dan $|11\rangle$.

$$(\rho_A)_{11} = \rho_{10,10} + \rho_{11,11} = \rho[2, 2] + \rho[3, 3]$$

Ini juga sesuai dengan kode Anda!

Untuk elemen off-diagonal:

$(\rho_A)_{01}$ (di basis $|0\rangle\langle 1|$):

Keadaan awal: $|0\rangle_A \otimes (\dots)$ dan $|1\rangle_A \otimes (\dots)$.

$$(\rho_A)_{01} = \rho_{00,10} + \rho_{01,11} = \rho[0, 2] + \rho[1, 3]$$

Cocok dengan kode Anda!

$(\rho_A)_{10}$ adalah complex conjugate dari $(\rho_A)_{01}$:

$$(\rho_A)_{10} = \rho_{10,00} + \rho_{11,01} = \rho[2, 0] + \rho[3, 1]$$

Cocok dengan kode Anda!

Kesimpulan: Kode Anda secara tepat mengimplementasikan operasi **Partial Trace**

$$\rho_A = \text{Tr}_B[\rho_{AB}].$$

2.4. Menguji Pemahaman dengan Contoh Kunci

Untuk memperdalam intuisi, mari kita hitung ρ_A untuk dua contoh penting: keadaan produk dan keadaan teralitkan maksimal. Ini akan menjadi jembatan ke Tahap 3 (Entropi Von Neumann).

Contoh 1: Keadaan Produk $|\Psi\rangle_{AB} = \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)$

Matriks densitasnya sudah kita hitung di atas:

$$\rho_{AB} = \frac{1}{4} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix}$$

Sekarang kita hitung ρ_A :

- $\rho_A[0, 0] = \rho[0, 0] + \rho[1, 1] = 1/4 + 1/4 = 1/2$
- $\rho_A[1, 1] = \rho[2, 2] + \rho[3, 3] = 1/4 + 1/4 = 1/2$
- $\rho_A[0, 1] = \rho[0, 2] + \rho[1, 3] = 1/4 + 1/4 = 1/2$
- $\rho_A[1, 0] = \rho[2, 0] + \rho[3, 1] = 1/4 + 1/4 = 1/2$

$$\text{Jadi, } \rho_A = \begin{pmatrix} 1/2 & 1/2 \\ 1/2 & 1/2 \end{pmatrix}.$$

Ini adalah matriks densitas yang **murni (pure)**. Kita tahu ini karena bisa ditulis sebagai

$\rho_A = |+\rangle\langle +|$, dengan $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$. Sesuai harapan, qubit A dalam keadaan

superposisi murni, tidak ada keteralitan dengannya.

Contoh 2: Keadaan Teralitkan Maksimal $|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$

Statevectornya: $\frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 0 \\ 0 \\ 1 \end{pmatrix}$

Matriks densitas ρ_{AB} adalah:

$$\rho_{AB} = \frac{1}{2} \begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 \end{pmatrix}$$

Sekarang hitung ρ_A :

- $\rho_A[0, 0] = \rho[0, 0] + \rho[1, 1] = 1/2 + 0 = 1/2$
- $\rho_A[1, 1] = \rho[2, 2] + \rho[3, 3] = 0 + 1/2 = 1/2$
- $\rho_A[0, 1] = \rho[0, 2] + \rho[1, 3] = 0 + 0 = 0$
- $\rho_A[1, 0] = \rho[2, 0] + \rho[3, 1] = 0 + 0 = 0$

Jadi, $\rho_A = \begin{pmatrix} 1/2 & 0 \\ 0 & 1/2 \end{pmatrix}$.

Ini adalah perbedaan krusial! Matriks densitas ini adalah matriks identitas yang dikalikan 1/2. Ia merepresentasikan **keadaan campuran maksimal (maximally mixed state)**. Qubit A tidak lagi berada dalam superposisi murni $|+\rangle$, melainkan dalam campuran statistik klasik: 50% probabilitas $|0\rangle$ dan 50% probabilitas $|1\rangle$.

Implikasinya: Keteralitan telah "menghancurkan" koherensi murni subsistem. Semakin teralitkan suatu sistem, semakin tidak murni (semakin campuran) subsistemnya jika dilihat secara terpisah. **Inilah kunci yang menghubungkan Partial Trace ke Entropi Von Neumann di Tahap 3.**

Anda telah menyelesaikan **Tahap 2** dengan solid. Anda kini memahami:

1. Konsep Matriks Densitas $\rho = |\psi\rangle\langle\psi|$.
2. Operasi Partial Trace $\rho_A = \text{Tr}_B[\rho_{AB}]$ dan bagaimana ia "membuang" informasi dari sistem lain.
3. Bagaimana kode Python Anda secara eksak menjumlahkan elemen matriks yang tepat untuk menghitung Partial Trace.
4. Perbedaan mendasar antara ρ_A dari keadaan produk (sebuah pure state) dan ρ_A dari keadaan teralitkan (sebuah mixed state).

Kita sekarang siap untuk melangkah ke **Tahap 3: Definisi dan Formula Entropi Von Neumann**, di mana kita akan mengubah matriks densitas tereduksi ini menjadi sebuah angka ajaib yang mengukur derajat keteralitan. Apakah Anda siap?

Baik, kita masuk ke **Tahap 3: Definisi dan Formula Entropi Von Neumann**.

Inilah puncak dari seluruh persiapan matematis kita. Kita sudah memiliki:

- Statevector 2-qubit ($|\psi\rangle$, vektor 4D)
- Matriks densitas tereduksi untuk qubit A (ρ_A , matriks 2x2) dari Tahap 2

Sekarang kita akan mengubah matriks ρ_A menjadi **satu angka skalar**, yaitu **Entropi Von Neumann** $S(\rho_A)$, yang mengukur seberapa teralitkan qubit A dengan qubit B.

3.1. Motivasi dari Entropi Klasik: Shannon Entropy

Di dunia klasik, misalkan kita punya sumber informasi yang menghasilkan simbol x_i dengan probabilitas p_i . **Entropi Shannon** mengukur rata-rata "kejutan" atau ketidakpastian dari sumber itu:

$$H(X) = - \sum_i p_i \log_2(p_i)$$

- Jika sumber **deterministik** (misal $p_1 = 1, p_2 = 0, \dots$), maka entropinya $H = 0$. Tidak ada ketidakpastian sama sekali.
- Jika sumber **seragam** (misal 4 simbol dengan $p_i = 1/4$), maka entropinya maksimal:
 $H = -4 \times (1/4) \times \log_2(1/4) = 2\text{bit}$.

Di dunia kuantum, kita akan mengganti "distribusi probabilitas p_i " dengan "matriks densitas ρ ", dan jumlah biasa dengan **trace**. Ini menghasilkan definisi yang sangat mirip.

3.2. Definisi Formal Entropi Von Neumann

Entropi Von Neumann $S(\rho)$ dari sebuah matriks densitas ρ didefinisikan sebagai:

$$S(\rho) = -\text{Tr}[\rho \log_2(\rho)]$$

di mana $\log_2$ adalah logaritma basis 2, dan Tr adalah trace (jumlah elemen diagonal).

Namun, bagaimana cara menghitungnya secara praktis?

Kita tidak bisa langsung memasukkan matriks ke dalam fungsi log. Kita harus mendiagonalisasi ρ terlebih dahulu. Setiap matriks densitas adalah matriks Hermitian ($\rho = \rho^\dagger$), sehingga ia dapat didiagonalisasi oleh matriks uniter U :

$$\rho = U\Lambda U^\dagger$$

Di mana $\Lambda = \text{diag}(\lambda_1, \lambda_2, \dots)$ adalah matriks diagonal yang berisi **nilai eigen (eigenvalues)** λ_i dari ρ .

Karena $\log_2(\rho) = U \log_2(\Lambda) U^\dagger$, maka:

$$S(\rho) = -\text{Tr}[U\Lambda U^\dagger \cdot U \log_2(\Lambda) U^\dagger] = -\text{Tr}[\Lambda \log_2(\Lambda)]$$

di mana $\log_2(\Lambda) = \text{diag}(\log_2(\lambda_1), \log_2(\lambda_2), \dots)$.

Karena trace tidak berubah terhadap perubahan basis uniter, kita mendapatkan **rumus kerja yang sangat sederhana**:

$$S(\rho) = - \sum_i \lambda_i \log_2(\lambda_i)$$

di mana λ_i adalah **nilai-nilai eigen dari matriks densitas** ρ .

Ini adalah jantung dari kode Python Anda. Entropi Von Neumann hanyalah **Entropi Shannon** dari nilai-nilai eigen matriks densitas.

3.3. Menghubungkan Rumus dengan Kode Anda

Mari kita bedah fungsi `calculate_entanglement_entropy(psi)` di **halaman 8** baris per baris, dengan pemahaman baru ini.

```
def calculate_entanglement_entropy(psi):
    """Menghitung Entanglement Entropy dari statevector 2-qubit."""

    # 1. Matriks Densitas Gabungan (rho_AB)
    rho = np.outer(psi, np.conj(psi))
```

Ini adalah $\rho_{AB} = |\psi\rangle\langle\psi|$, matriks 4x4 dari seluruh sistem.

```
    # 2. Partial Trace untuk Mendapatkan Matriks Densitas Tereduksi
    (rho_A)
    rho_A = np.zeros((2, 2), dtype=complex)
    rho_A[0, 0] = rho[0, 0] + rho[1, 1]
    rho_A[1, 1] = rho[2, 2] + rho[3, 3]
    rho_A[0, 1] = rho[0, 2] + rho[1, 3]
    rho_A[1, 0] = rho[2, 0] + rho[3, 1]
```

Ini adalah operasi $\rho_A = \text{Tr}_B[\rho_{AB}]$ yang sudah kita kuasai di Tahap 2. Hasilnya matriks 2x2.

```
    # 3. Menghitung Nilai Eigen dari rho_A
    eigvals = np.linalg.eigvalsh(rho_A)
```

`np.linalg.eigvalsh` adalah fungsi Python untuk menghitung nilai eigen dari matriks Hermitian. Karena ρ_A dijamin Hermitian, nilai-nilai eigennya (λ_1, λ_2) adalah bilangan real. Ini adalah "probabilitas kuantum" kita.

```
# 4. Membersihkan Nilai Negatif Sangat Kecil Akibat Floating Point
eigvals = eigvals[eigvals > 1e-12]
```

Ini langkah praktis. Secara teori, nilai eigen λ_i selalu ≥ 0 . Karena keterbatasan presisi komputer, bisa muncul angka seperti -10^{-16} . Kita buang nilai-nilai ini agar tidak error saat dihitung $\log_2$.

```
# 5. Von Neumann Entropy: S = -sum(p * log2(p))
return -np.sum(eigvals * np.log2(eigvals))
```

Ini persis seperti $S(\rho) = -\sum_i \lambda_i \log_2(\lambda_i)$. Kode Anda menghitung Entropi Shannon dari nilai eigen ρ_A , dan itulah Entropi Von Neumann.

3.4. Analisis Dua Contoh Kunci (Dari Tahap 2)

Sekarang, mari kita hitung entropi untuk dua contoh kita dan lihat bagaimana ia menangkap esensi keteralitan.

Contoh 1: Keadaan Produk $|\Psi\rangle_{AB} = |+\rangle \otimes |+\rangle$

Dari Tahap 2, kita dapat:

$$\rho_A = \begin{pmatrix} 1/2 & 1/2 \\ 1/2 & 1/2 \end{pmatrix}$$

Untuk mencari nilai eigen, selesaikan $\det(\rho_A - \lambda I) = 0$:

$$\det \begin{pmatrix} 1/2 - \lambda & 1/2 \\ 1/2 & 1/2 - \lambda \end{pmatrix} = (\lambda - 1)\lambda = 0$$

Jadi, $\lambda_1 = 1$ dan $\lambda_2 = 0$.

Maka entropinya:

$$S(\rho_A) = -(1 \cdot \log_2(1) + 0 \cdot \log_2(0))$$

Dengan definisi limit, $0 \cdot \log_2(0) = 0$.

$$S(\rho_A) = 0$$

Interpretasi: Entropi Von Neumann untuk subsistem dari keadaan produk adalah **NOL**.

Tidak ada keteralitan. Jika kita hanya mengamati qubit A, ia berada dalam keadaan murni $|+\rangle$ yang sepenuhnya koheren. Tidak ada ketidakpastian kuantum (campuran) yang disebabkan oleh keteralitan.

Contoh 2: Keadaan Teralitkan Maksimal $|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$

Dari Tahap 2, kita dapat:

$$\rho_A = \begin{pmatrix} 1/2 & 0 \\ 0 & 1/2 \end{pmatrix}$$

Matriks ini sudah diagonal. Nilai eigennya jelas:

$$\lambda_1 = 1/2, \quad \lambda_2 = 1/2$$

Maka entropinya:

$$S(\rho_A) = -\left(\frac{1}{2}\log_2\left(\frac{1}{2}\right) + \frac{1}{2}\log_2\left(\frac{1}{2}\right)\right) = -(\log_2\left(\frac{1}{2}\right)) = -(-1) = 1$$

$$\boxed{S(\rho_A) = 1}$$

Interpretasi: Entropi Von Neumann untuk subsistem dari keadaan teralitkan maksimal adalah **SATU**. Ini adalah nilai maksimum untuk sistem 2-qubit. Qubit A, jika diamati sendiri, terlihat seperti keadaan campuran maksimal (50%|0>, 50%|1>). Semua koherensinya "hilang" ke dalam korelasinya dengan qubit B. Inilah "kualitas" keteralitan yang diukur oleh entropi.

3.5. Rangkuman Tahap 3

Anda sekarang telah menguasai inti dari alat ukur Anda:

1. **Rumus Kerja:** $S(\rho_A) = -\sum_i \lambda_i \log_2(\lambda_i)$, di mana λ_i adalah nilai eigen dari matriks densitas tereduksi ρ_A .
2. **Koneksi Kode:** Anda dapat melihat dengan jelas bagaimana setiap baris kode Python menerjemahkan rumus matematika ini, dari membangun ρ , partial trace, hingga diagonalisasi.
3. **Arti Fisik:**
 - $S = 0$: Subsistem dalam keadaan murni. **Tidak ada keteralitan.** (Product state)
 - $0 < S < 1$: **Teralitkan sebagian.**
 - $S = 1$: **Teralitkan maksimal.** (Keadaan Bell)

Kita telah membangun fondasi yang sangat kuat. Dengan pemahaman ini, kita siap untuk naik ke level yang lebih tinggi: **Tahap 4: Ansatz dan Masalah Optimasi VQE**. Di tahap itu, kita akan melihat bagaimana sirkuit kuantum yang Anda gunakan secara sistematis menaikkan dan menurunkan entropi ini, dan bagaimana ia merepresentasikan masalah finansial Anda.

Apakah Anda siap melanjutkan ke **Tahap 4**?

Baik, kita lanjut ke **Tahap 4: Ansatz dan Masalah Optimasi VQE**.

Di tiga tahap sebelumnya, kita telah membangun pemahaman fundamental: bagaimana sistem 2-qubit direpresentasikan (statevector 4D), bagaimana kita mengisolasi subsistem (partial trace $\rightarrow \rho_A$), dan bagaimana kita mengukur keteralitan subsistem itu (entropi Von Neumann $S(\rho_A)$).

Sekarang kita akan masuk ke inti algoritma Anda: **Variational Quantum Eigensolver (VQE)**. Kita akan membahas dua komponen utamanya:

1. **Ansatz**: Sirkuit kuantum parametrik yang membangun statevector $|\psi(\theta)\rangle$.
2. **Masalah Optimasi**: Bagaimana masalah portofolio Markowitz dikodekan ke dalam Hamiltonian, sehingga energi $\langle\psi(\theta)|H|\psi(\theta)\rangle$ menjadi fungsi biaya yang harus diminimalkan.

4.1. Arsitektur Sirkuit (Ansatz) VQE Anda

Ansatz adalah "hipotesis" kita tentang bentuk fungsi gelombang. Dalam VQE, ansatz adalah sirkuit kuantum dengan parameter yang bisa di-*tuning*. Tujuannya: menemukan kombinasi parameter θ yang menghasilkan statevector $|\psi(\theta)\rangle$ dengan energi terendah terhadap Hamiltonian H .

Mari kita bedah fungsi `get_psi_theta(theta, depth)` di **halaman 4-5 dan 16** file Anda. Sirkuit Anda terdiri dari dua jenis blok yang diulang:

A. Blok Rotasi (Rotation Layer)

Untuk setiap qubit $q \in \{0, 1\}$ pada setiap layer $i \in \{0, \dots, \text{depth} - 1\}$, kita menerapkan dua gerbang rotasi:

$$U_{q,i}(\theta) = R_Z(\theta_{i,q,1}) \cdot R_Y(\theta_{i,q,0})$$

Di mana:

$$R_Y(\phi) = \begin{pmatrix} \cos(\phi/2) & -\sin(\phi/2) \\ \sin(\phi/2) & \cos(\phi/2) \end{pmatrix}$$

$$R_Z(\phi) = \begin{pmatrix} e^{-i\phi/2} & 0 \\ 0 & e^{i\phi/2} \end{pmatrix}$$

Ini adalah gerbang 1-qubit yang memutar statevector individual di Bloch sphere. Rotasi ini **tidak menciptakan keteralitan**; mereka hanya mengubah superposisi lokal masing-masing qubit.

Untuk dua qubit, operator layer rotasi adalah produk tensor (Kronecker):

$$U_{\text{layer},i} = U_{0,i} \otimes U_{1,i}$$

Kode Anda melakukan ini dengan:

```
u0 = rz(theta[i, 0, 1]) @ ry(theta[i, 0, 0])
u1 = rz(theta[i, 1, 1]) @ ry(theta[i, 1, 0])
u_layer = np.kron(u0, u1) # atau manual_kron_2x2 di versi njit
psi = u_layer @ psi
```

B. Blok Entanglement (Entangling Layer)

Setelah blok rotasi, kita menerapkan gerbang dua-qubit yang menciptakan keterkaitan. Sirkuit Anda menggunakan dua gerbang CNOT secara berurutan:

1. **CNOT01**: Kontrol = Q0, Target = Q1
2. **CNOT10**: Kontrol = Q1, Target = Q0

Secara matematis, ini adalah operator uniter $U_{\text{entang}} = \text{CNOT}_{10} \cdot \text{CNOT}_{01}$.

Kode Anda:

```
if i < depth - 1:
    psi = cnot01 @ psi
    psi = cnot10 @ psi
```

Mengapa dua CNOT dua arah?

Satu CNOT menciptakan keterkaitan, tetapi dengan arah kontrol yang tetap ($Q0 \rightarrow Q1$). Dengan menerapkan CNOT dua arah ($Q0 \rightarrow Q1$ lalu $Q1 \rightarrow Q0$), kita menciptakan struktur keterkaitan yang lebih simetris antara kedua qubit.

Struktur Lengkap Sirkuit:

Untuk `depth = 6`, sirkuit Anda adalah:

$$|\psi(\theta)\rangle = \underbrace{U_{\text{layer},6}}_{\text{Rotasi}} \cdot \underbrace{U_{\text{entang}} \cdot U_{\text{layer},5} \cdots U_{\text{entang}} \cdot U_{\text{layer},2}}_{5 \text{ blok penuh}} \cdot \underbrace{U_{\text{layer},1}}_{\text{Rotasi}} |00\rangle$$

Perhatikan: entanglement TIDAK diterapkan setelah layer rotasi terakhir. Ini karena kita ingin layer terakhir melakukan fine-tuning superposisi lokal tanpa mengubah struktur keterkaitan yang sudah terbentuk.

Parameterisasi:

Setiap layer memiliki $2 \text{ qubit} \times 2 \text{ rotasi} = 4$ parameter. Untuk `depth` d , total parameter adalah $4d$. Dalam kode Anda, $d = 7$ (lihat `params = np.random.rand(7, 2, 2)` di halaman 5), sehingga ada 28 parameter. Semua parameter ini adalah sudut rotasi $\theta \in [0, 2\pi]$.

4.2. Fungsi Tujuan (Cost Function) Markowitz dan Hamiltonian Kuantum

Sekarang kita beralih ke masalah yang ingin diselesaikan: **Optimasi Portofolio Markowitz**.

A. Masalah Klasik

Dalam teori portofolio modern, kita ingin memilih bobot investasi w (vektor berisi persentase alokasi ke setiap aset) yang memaksimalkan expected return $\mu^T w$ sambil meminimalkan risiko $w^T \Sigma w$, di mana Σ adalah matriks kovarians return aset.

Ini biasanya dirumuskan sebagai minimisasi fungsi biaya:

$$\mathcal{L}(w) = \gamma \cdot w^T \Sigma w - \mu^T w$$

dengan kendala $\sum_i w_i = 1$, di mana γ adalah koefisien risk aversion.

B. Pengkodean Kuantum (Quantum Encoding)

Untuk menyelesaikan ini di komputer kuantum, kita harus mengkodekan bobot portofolio w ke dalam state vector kuantum $|\psi(\theta)\rangle$.

Ada beberapa cara. Yang paling umum untuk VQE adalah **amplitude encoding**:

$$w_i = |\langle i | \psi(\theta) \rangle|^2$$

Di mana $|i\rangle$ adalah basis komputasi (untuk 2 aset: $|00\rangle, |01\rangle, |10\rangle, |11\rangle$).

Jadi, probabilitas mengukur setiap basis state menjadi bobot alokasi aset. Kendala $\sum w_i = 1$ otomatis terpenuhi karena total probabilitas = 1.

C. Hamiltonian

Tujuannya adalah merumuskan $\mathcal{L}(w)$ sebagai nilai ekspektasi kuantum $\langle \psi(\theta) | H | \psi(\theta) \rangle$.

Untuk amplitudo encoding, Hamiltonian H adalah matriks 4x4 yang elemennya diturunkan dari Σ dan μ .

Dari kode Anda di halaman 2:

```
returns = df_last_year.drop(columns='Date').pct_change().dropna()
mu = returns.mean().values
sigma_matrix = returns.cov().values
gamma = 0.5
```

Anda menghitung expected return μ dan matriks kovarians Σ dari data historis saham BBCA.JK dan TLKM.JK. Kemudian Hamiltonian H dibangun dari data ini (proses konstruksinya tidak ditampilkan secara eksplisit di file, tetapi hasilnya H digunakan di halaman 6 dan seterusnya).

Untuk 2 aset, H adalah matriks 4x4 yang memenuhi:

$$\langle \psi | H | \psi \rangle = \gamma \cdot (\text{varians portofolio}) - (\text{expected return})$$

4.3. Mengapa Depth yang Tepat Sangat Penting: Kaitannya dengan Entropi

Di sinilah kita mulai melihat bayangan jawaban dari misteri utama. Mengapa grafik energi di halaman 15 menunjukkan performa terbaik di depth 8-12, tetapi hasil uji konsistensi di halaman 19-20 menunjukkan depth 4-6 lebih stabil?

Depth Rendah (1-3): Ekspresifitas Terbatas

- Ansatz dengan sedikit layer rotasi dan entanglement hanya bisa menghasilkan statevector dengan struktur keteralitan sederhana.
- **Entropi rendah**, variasi state yang bisa dijangkau sedikit.
- Akibatnya, solusi optimal mungkin tidak bisa direpresentasikan. Energi minimum yang dicapai masih tinggi.

Depth Optimal (4-6): Sweet Spot

- Ansatz cukup dalam untuk menghasilkan berbagai konfigurasi keteralitan.
- **Entropi cukup tinggi** untuk ekspresifitas, tetapi **tidak terlalu tinggi** hingga jatuh ke dalam zona Barren Plateau (akan dijelaskan di Tahap 5).
- Ruang pencarian cukup kaya untuk menemukan solusi mendekati optimal, tetapi gradien masih cukup besar untuk navigasi yang efektif.

Depth Tinggi (8-12): Over-Parameterisasi dan Barren Plateau

- Ansatz terlalu ekspresif. Statevector bisa mencapai hampir semua titik di ruang Hilbert secara acak.
- **Entropi tinggi** (mendekati 1 untuk banyak parameter acak).
- Ini memicu **Barren Plateau**: variansi gradien menurun secara eksponensial, membuat optimasi menjadi sangat sulit (akan dijelaskan detail di Tahap 5).

4.4. Ringkasan Tahap 4

Anda sekarang memahami:

1. **Ansatz** Anda sebagai rangkaian berulang: Rotasi (lokal, tanpa entangle) → Entanglement (dua arah, menciptakan korelasi kuantum) → Rotasi → ...
2. **Parameterisasi**: 28 parameter sudut rotasi yang mengontrol evolusi state.
3. **Encoding**: Amplitudo statevector $|\psi(\theta)\rangle$ merepresentasikan bobot portofolio w_i .
4. **Fungsi Biaya**: $\langle\psi|H|\psi\rangle$ adalah energi yang harus diminimalkan; H berisi informasi expected return dan matriks kovarians dari data saham.
5. Munculnya dugaan hubungan antara **depth, entropi, dan performa optimasi**: depth rendah (entropi rendah, ekspresifitas terbatas), depth optimal (entropi sedang, trainabilitas baik), depth tinggi (entropi tinggi, potensi Barren Plateau).

Kita sekarang berada di ambang untuk memahami misteri utama: **Mengapa entropi Von Neumann dapat mendeteksi Barren Plateau?**

Siapa melanjutkan ke **Tahap 5: Fenomena Barren Plateau**? Di sana kita akan mendefinisikan secara matematis monster yang menghantui VQE, dan bagaimana ia terkait dengan variansi gradien yang Anda ukur di kode. Siapa? Kita memasuki **Tahap 5: Fenomena Barren Plateau (Dataran Tandus)**.

Ini adalah tahap di mana kita akan mendefinisikan secara matematis "monster" yang menjadi alasan utama mengapa Anda menghitung entropi Von Neumann dan variansi gradien. Di sinilah semua benang merah mulai terikat.

5.1. Apa Itu Barren Plateau? Definisi Formal

Barren Plateau adalah fenomena dalam komputasi kuantum variasional di mana **variansi dari gradien fungsi biaya menurun secara eksponensial terhadap jumlah qubit n** .

Secara matematis, untuk ansatz parametrik acak dengan parameter θ_k , gradien dari nilai ekspektasi $\langle H \rangle$ terhadap parameter tersebut memenuhi:

$$\text{Var}_{\theta}[\partial_k \langle H \rangle] \propto \frac{1}{2^n}$$

Atau secara lebih umum, untuk keluarga ansatz tertentu:

$$\text{Var}[\partial_k E] \in \mathcal{O}\left(\frac{1}{b^n}\right)$$

dengan $b > 1$ dan n adalah jumlah qubit.

Apa artinya ini dalam praktek?

- Gradien adalah "sinyal" yang memberi tahu algoritma optimasi (seperti SPSA di kode Anda) ke arah mana parameter harus diubah untuk menurunkan energi.
- Jika variansi gradien $\rightarrow 0$, maka hampir semua nilai gradien terkonsentrasi di sekitar nol dengan deviasi yang sangat kecil.
- Akibatnya: **Algoritma optimasi tidak bisa membedakan arah yang baik dari arah yang buruk**. Ia berjalan di permukaan yang hampir datar sempurna (itulah "dataran tandus"), dan jumlah iterasi yang dibutuhkan untuk menemukan minimum meledak secara eksponensial.
- Untuk sistem dengan $n > 10$, ini bisa membuat VQE hampir mustahil berhasil tanpa strategi khusus.

5.2. Mengapa Barren Plateau Terjadi? Hubungan dengan Ansatz Acak

Penyebab utama Barren Plateau adalah **ansatz yang terlalu ekspresif sehingga berperilaku seperti desain acak (random unitary)**.

Mari kita bedah logikanya:

1. Ansatz sebagai Peta dari Parameter ke State:

Ansatz Anda adalah fungsi $|\psi(\theta)\rangle = U(\theta)|0\rangle^{\otimes n}$, di mana $U(\theta)$ adalah produk dari gerbang-gerbang rotasi dan entanglement.

2. Inisialisasi Acak:

Saat Anda memulai optimasi, Anda menginisialisasi θ secara acak (di kode Anda:

```
theta_k = np.random.uniform(0, 2*np.pi, 28) ).
```

3. Ansatz yang Sangat Dalam → Distribusi State yang Seragam:

Jika rangkaian Anda cukup dalam (banyak layer rotasi dan entanglement), maka untuk θ acak, state $|\psi(\theta)\rangle$ akan terdistribusi hampir **seragam** di seluruh ruang Hilbert (menurut Haar measure).

Artinya, $|\psi(\theta)\rangle$ bisa menjadi state kuantum **apa saja** dengan probabilitas yang hampir sama.

4. Konsekuensi untuk Gradien:

Jika state terdistribusi seragam, maka nilai ekspektasi $\langle\psi|H|\psi\rangle$ untuk θ acak akan terkonsentrasi di sekitar nilai rata-rata global. Fluktuasi terhadap parameter menjadi sangat kecil.

Secara intuitif: Jika Anda berada di titik acak di permukaan bola dimensi tinggi, dan Anda mengambil langkah kecil ke arah acak, perubahan ketinggian Anda hampir selalu ≈ 0 .

Tidak ada "lereng" yang bisa diikuti.

5.3. Indikator Kunci: Variansi Gradien

Inilah mengapa di kode Anda, tepatnya di **halaman 9**, Anda menghitung:

```
# 1. Track Variansi Gradien untuk deteksi Barren Plateaus
grad_variance = np.var(g_k)
```

Di sini, g_k adalah estimasi gradien dari algoritma SPSA untuk satu perturbasi:

$$g_k = \frac{E(\theta_k + c_k \Delta_k) - E(\theta_k - c_k \Delta_k)}{2c_k} \Delta_k^{-1}$$

Anda menghitung **variansi dari komponen-komponen vektor gradien** g_k pada setiap iterasi. Jika ansatz Anda memasuki zona Barren Plateau, nilai `grad_variance` ini akan **sangat kecil** (misalnya, 10^{-8} atau lebih kecil).

Dari grafik yang Anda hasilkan di halaman 13, Anda bisa melihat bagaimana `Var[Grad]` berubah terhadap `depth`. Untuk `depth` yang sangat tinggi (8, 9, 10, 11, 12), nilai variansi gradien ini kemungkinan besar turun drastis, menunjukkan bahwa optimasi menjadi sangat sulit.

5.4. Koneksi Pertama dengan Entropi Von Neumann

Sekarang kita mulai melihat hubungannya:

- **Ansatz Acak + Dalam** → State $|\psi(\theta)\rangle$ hampir pasti **teralitkan maksimal**.
- **Teralitkan Maksimal** → Matriks densitas tereduksi ρ_A mendekati $\frac{1}{2}I$ (keadaan campuran maksimal).
- **Keadaan Campuran Maksimal** → Semua nilai eigen $\lambda_i \approx 1/2 \rightarrow$ **Entropi Von Neumann** $S \approx 1$ (nilai maksimum).

Dan di saat yang sama:

- **Ansatz Acak + Dalam** → State terdistribusi seragam → **Variansi Gradien** $\text{Var}[\partial_k E] \rightarrow 0$.

Ini hipotesis yang mendasari eksperimen Anda:

Semakin tinggi entropi Von Neumann rata-rata dari state yang dihasilkan ansatz acak, semakin besar kemungkinan ansatz tersebut mengalami Barren Plateau (variansi gradien kecil).

Atau dengan kata lain: **Entropi Von Neumann dapat berfungsi sebagai "early warning system" untuk Barren Plateau.**

5.5. Mengapa Depth 4,5,6 Menjadi Kandidat Terbaik?

Mari kita lihat hasil evaluasi Anda di halaman 19:

	Depth	Mean Energy	Std Dev
0	4	-0.000171	0.000911
1	5	-0.000638	0.000063
2	6	-0.000648	0.000061

- **Depth 4:** Mean energy masih relatif tinggi (-0.000171) dan **Std Dev sangat besar** (0.000911). Ini menunjukkan ansatz belum cukup stabil; kadang menemukan solusi baik, kadang buruk. Ekspresifitas mungkin masih kurang.
- **Depth 5 dan 6:** Mean energy jauh lebih rendah (sekitar -0.00064) dan **Std Dev jauh lebih kecil** ($\sim 6 \times 10^{-5}$). Ini adalah **sweet spot**: ansatz cukup ekspresif untuk secara konsisten menemukan solusi energi rendah, tetapi belum terlalu dalam hingga variansi gradien kolaps.

Apa yang terjadi di depth > 8?

Meskipun di grafik halaman 15 depth 8-12 mencapai energi yang lebih rendah, hasil ini mungkin tidak stabil atau tidak konsisten jika di-run ulang. Std Dev yang tidak ditampilkan untuk depth > 6 mungkin akan menunjukkan fluktuasi yang besar atau kegagalan konvergensi.

5.6. Ringkasan dan Jembatan ke Tahap 6

Anda sekarang memahami:

1. **Definisi Barren Plateau:** $\text{Var}[\partial_k E] \propto 1/2^n$, gradien hampir nol di mana-mana.
2. **Penyebab:** Ansatz yang terlalu dalam/ekspresif bertindak sebagai random unitary, mendistribusikan state secara seragam.
3. **Indikator:** Variansi gradien yang sangat kecil adalah tanda bahaya.
4. **Hubungan dengan Entropi:** State yang terdistribusi seragam cenderung menghasilkan keteralitan maksimal ($S \approx 1$), sehingga entropi tinggi berkorelasi dengan variansi gradien rendah (potensi Barren Plateau).

Kita sekarang siap untuk **Tahap 6: Jembatan Konsep** yang akan mensintesis semua ini menjadi satu narasi koheren, menganalisis grafik Anda, dan menyimpulkan mengapa entropi Von Neumann adalah alat diagnostik yang valid untuk memilih depth optimal.

Apakah Anda siap melanjutkan? Katakan "Lanjut Tahap 6".

Baik, kita sampai di **Tahap 6: Jembatan Konsep — Hubungan Entropi Von Neumann, Variansi Gradien, dan Barren Plateau**.

Ini adalah tahap sintesis. Kita akan menyatukan semua yang telah kita pelajari dari Tahap 1 sampai 5 menjadi satu narasi koheren, dan kemudian menggunakannya untuk **menganalisis grafik-grafik yang Anda hasilkan di file**. Inilah jawaban dari pertanyaan utama: "Bagaimana entropi Von Neumann membantu algoritma ini?"

6.1. Rumusan Formal Hubungan Entropi dan Barren Plateau

Dari literatur riset VQE (terutama karya-karya tentang *expressibility*, *entanglement*, dan *trainability*), terdapat hubungan kualitatif dan kuantitatif yang kuat:

Hipotesis Inti:

Untuk sebuah ansatz $U(\theta)$ dengan depth d dan jumlah qubit n :

1. **Ekspresifitas** ansatz dapat diukur dengan seberapa seragam ia mendistribusikan state di ruang Hilbert. Semakin tinggi depth, semakin tinggi ekspresifitasnya, hingga akhirnya mendekati *Haar random unitary* (desain acak).
2. **Ekspresifitas yang terlalu tinggi (mendekati acak) $\Rightarrow$ Barren Plateau:**

$$\text{Var}[\partial_k E] \propto \frac{1}{2^n}$$

3. **Entropi Von Neumann rata-rata** dari subsistem untuk state yang dihasilkan oleh parameter acak θ juga meningkat seiring ekspresifitas ansatz:

$$\mathbb{E}_{\theta}[S(\rho_A(\theta))] \xrightarrow{\text{depth besar}} \log_2(d_A) = 1 \quad (\text{untuk } d_A = 2)$$

Di mana d_A adalah dimensi subsistem A. Untuk satu qubit, $d_A = 2$, jadi batas maksimumnya $\log_2(2) = 1$.

Kesimpulan Teoretis:

Keduanya (variansi gradien dan entropi Von Neumann) adalah **dua sisi dari mata uang yang sama**: keduanya merupakan konsekuensi dari ansatz yang terlalu ekspresif/mendekati desain acak.

- **Entropi Von Neumann** $S \rightarrow 1 \iff$ subsistem terlihat seperti campuran maksimal $\iff$ state global terdistribusi seragam.
- **Distribusi seragam** $\iff$ tidak ada arah preferensial di lanskap energi $\iff$ **Variansi Gradien** $\text{Var}[\partial_k E] \rightarrow 0$.

Dengan demikian, entropi Von Neumann dapat berfungsi sebagai proksi (pengganti) untuk mendeteksi Barren Plateau TANPA harus menghitung variansi gradien secara eksplisit.

6.2. Analisis Grafik Anda: Membaca "Termometer Ekspresifitas"

Sekarang kita akan menafsirkan grafik yang Anda hasilkan di **halaman 12 dan 13**, menggunakan lensa teoretis di atas.

Anggaplah grafik-grafik itu menunjukkan pola berikut (berdasarkan apa yang kita duga dari hasil akhir Anda):

A. Grafik Entropi Von Neumann vs. Iterasi (Halaman 12)

- **Depth 1-3 (Rendah):**
 - Entropi mungkin berfluktuasi di sekitar 0.2 - 0.5.
 - **Interpretasi:** Ansatz belum cukup ekspresif. Struktur keteralitan yang bisa dibentuk terbatas. State cenderung dekat dengan *product state*. Ini adalah zona "kurang matang".
- **Depth 4-6 (Optimal):**
 - Entropi mungkin berada di kisaran 0.5 - 0.8.
 - **Interpretasi:** Inilah *sweet spot*. Ansatz cukup kaya untuk menghasilkan berbagai tingkat keteralitan, tetapi belum sepenuhnya acak. Gradien masih memiliki variansi yang cukup untuk navigasi optimasi. Ini adalah zona "matang sempurna".
- **Depth 8-12 (Tinggi):**

- Entropi akan dengan cepat melonjak dan menetap di dekat **1.0**, mungkin di atas 0.95.
- **Interpretasi:** Ansatz terlalu ekspresif. Hampir semua parameter acak menghasilkan state yang hampir teralitkan maksimal. Subsystem qubit tunggalnya mendekati *maximally mixed state* $\frac{1}{2}I$. Ini adalah zona "terlalu matang" atau "gosong".

B. Grafik Variansi Gradien vs. Iterasi (Halaman 13)

- **Depth 1-3 (Rendah):**
 - Variansi gradien mungkin terlihat "normal" atau bahkan tinggi, tetapi fluktuatif.
 - **Interpretasi:** Gradien ada, tetapi karena ekspresifitas terbatas, mungkin iterasinya tidak konvergen ke solusi yang sangat baik.
- **Depth 4-6 (Optimal):**
 - Variansi gradien mungkin masih cukup besar (misal $\sim 10^{-4}$ atau 10^{-5}), menunjukkan sinyal gradien yang sehat.
 - **Interpretasi:** SPSA memiliki "medan" yang cukup curam untuk dilalui menuju minimum.
- **Depth 8-12 (Tinggi):**
 - Variansi gradien akan **kolaps** ke nilai yang sangat kecil, misalnya 10^{-8} atau 10^{-10} (seperti 10^{-2e} atau serupa).
 - **Interpretasi:** Inilah **fingerprint Barren Plateau**. Permukaan energi hampir datar sempurna. SPSA berjalan di tempat. Meskipun di beberapa run mungkin "kebetulan" menemukan solusi baik, prosesnya sangat tidak stabil dan tidak reproduisibel.

6.3. Tabel Kebenaran: Membaca Diagnostik

Kita bisa meringkas hubungan diagnostik ini dalam sebuah tabel:

Kondisi	Entropi Von Neumann(S)	Variansi Gradien $\text{Var}[g]$	Status Ansatz	Implikasi untuk VQE
Underfitting	Rendah ($S < 0.5$)	Normal/Tinggi	Kurang ekspresif	Tidak bisa mencapai solusi optimal secara konsisten (Mean Energy tinggi, Std Dev besar).
Sweet Spot	Sedang ($0.5 < S < 0.9$)	Sehat (cukup besar)	Ekspresif & Terlatih	Konvergensi stabil ke solusi energi rendah (Mean Energy rendah, Std Dev kecil).

Kondisi	Entropi Von Neumann(S)	Variansi Gradien $\text{Var}[g]$	Status Ansatz	Implikasi untuk VQE
Barren Plateau	Sangat Tinggi ($S \approx 1.0$)	Sangat Kecil ($\rightarrow 0$)	Terlalu ekspresif (acak)	Optimasi gagal atau sangat tidak stabil. Gradien hilang.

Data Anda di halaman 19:

	Depth	Mean Energy	Std Dev	
0	4	-0.000171	0.000911	-> Underfitting? (Std Dev besar)
1	5	-0.000638	0.000063	-> Sweet Spot!
2	6	-0.000648	0.000061	-> Sweet Spot!

Data ini **konsisten** dengan tabel di atas.

- Depth 4 menunjukkan energi rata-rata yang kurang baik dan Std Dev yang relatif besar. Mungkin ansatz di kedalaman ini masih berada di batas transisi menuju *sweet spot*.
- Depth 5 dan 6 berada persis di *sweet spot*. Ansatz cukup ekspresif untuk secara konsisten menemukan solusi dengan energi ≈ -0.00064 , dan sangat stabil (Std Dev kecil).

Untuk depth > 6 , meskipun grafik energi minimum (halaman 15) menunjukkan nilai yang lebih rendah, **konsistensi** diragukan. Tanpa menghitung Std Dev berulang, kita tidak tahu, tetapi berdasarkan teori Barren Plateau, depth yang lebih tinggi akan menunjukkan **Std Dev yang besar** atau bahkan gagal konvergen di banyak percobaan.

6.4. Mengapa Entropi Lebih Praktis? (Penting untuk Skala Besar)

Di sinilah letak kekuatan sejati entropi Von Neumann sebagai alat diagnostik:

1. **Kompleksitas:** Menghitung entropi Von Neumann untuk satu atau beberapa subsistem qubit adalah **jauh lebih murah** secara komputasi daripada menghitung seluruh vektor gradien pada banyak titik, apalagi menjalankan optimasi penuh berkali-kali.
2. **Prediktabilitas Pra-Pelatihan:** Anda bisa menghitung entropi Von Neumann rata-rata untuk beberapa set parameter acak θ SEBELUM menjalankan VQE yang mahal. Jika $\mathbb{E}[S]$ sudah mendekati 1, Anda tahu ansatz Anda mungkin terlalu dalam, dan Anda bisa mendesain ulang sebelum membuang sumber daya komputasi.
3. **Kemandirian dari Hamiltonian:** Entropi Von Neumann adalah properti dari **ansatz** itu sendiri, bukan dari Hamiltonian spesifik. Jadi, analisisnya berlaku untuk masalah optimasi apa pun yang menggunakan ansatz yang sama.

6.5. Ringkasan Tahap 6

Anda sekarang telah menyelesaikan jembatan konsep utama:

1. **Hubungan Teoretis:** Entropi Von Neumann yang tinggi ($S \approx 1$) dan Variansi Gradien yang rendah ($\text{Var}[g] \rightarrow 0$) adalah dua gejala dari penyebab yang sama: **ansatz yang terlalu ekspresif dan berperilaku seperti desain acak**.
2. **Diagnostik:** Entropi Von Neumann dapat digunakan sebagai **proksi** untuk mendeteksi kerentanan terhadap Barren Plateau.
3. **Analisis Hasil:** Hasil numerik Anda (Depth 5-6 optimal) sangat konsisten dengan kerangka teoretis ini.
4. **Keunggulan Praktis:** Entropi lebih murah dihitung dan bisa digunakan untuk skrining ansatz sebelum pelatihan.

Kita telah memahami *apa* hubungannya dan *bagaimana* membacanya. Sekarang kita siap untuk **Tahap 7: Peran dalam Algoritma Optimasi SPSA**, yang merupakan babak penutup untuk memahami bagaimana informasi ini secara praktis membantu algoritma Anda membuat keputusan.

Apakah Anda siap untuk menyelesaikan perjalanan ini dengan **Tahap 7?**

Kita sampai di penghujung roadmap: **Tahap 7: Peran dalam Algoritma Optimasi SPSA**.

Ini adalah tahap di mana semua teori yang kita bangun diterjemahkan menjadi keputusan praktis dalam algoritma Anda. Kita akan melihat bagaimana SPSA bekerja, mengapa ia rentan terhadap Barren Plateau, dan bagaimana pemantauan entropi Von Neumann membantu kita membuat pilihan desain yang cerdas **sebelum** dan **selama** optimasi.

7.1. Kilas Balik: Apa Itu Algoritma SPSA?

SPSA (Simultaneous Perturbation Stochastic Approximation) adalah algoritma optimasi stokastik yang dirancang untuk fungsi dengan dimensi tinggi, di mana menghitung gradien eksak tidak praktis atau tidak mungkin.

Di kode Anda, SPSA diimplementasikan di **halaman 8-9**. Mari kita lihat langkah-langkahnya.

Tujuan: Menemukan θ yang meminimalkan fungsi biaya $E(\theta) = \langle \psi(\theta) | H | \psi(\theta) \rangle$.

Langkah-langkah inti SPSA:

Misalkan θ_k adalah vektor parameter pada iterasi ke- k , dengan dimensi $p = 28$.

1. **Bangkitkan Vektor Perturbasi Acak Δ_k :**

$$\Delta_k = (\delta_1, \delta_2, \dots, \delta_p), \quad \delta_i \sim \text{Bernoulli}(\pm 1)$$

Di kode Anda: `delta_k = np.where(np.random.rand(28) > 0.5, 1, -1)`.

2. Evaluasi Fungsi di Dua Titik Terganggu:

$$E_+ = E(\theta_k + c_k \Delta_k)$$

$$E_- = E(\theta_k - c_k \Delta_k)$$

Di kode Anda: `theta_plus = theta_k + ck * delta_k` dan `theta_minus = theta_k - ck * delta_k`.

3. Estimasi Gradien (Satu Komponen):

Gradien parsial terhadap parameter ke- i diestimasi dengan:

$$\hat{g}_i = \frac{E_+ - E_-}{2c_k \delta_i}$$

Dalam bentuk vektor, ini adalah:

$$\hat{g}_k = \frac{E_+ - E_-}{2c_k} \Delta_k^{-1}$$

Di mana $\Delta_k^{-1} = (1/\delta_1, 1/\delta_2, \dots, 1/\delta_p)$. Karena $\delta_i \in \{\pm 1\}$, maka $\delta_i^{-1} = \delta_i$, sehingga:

$$\hat{g}_k = \frac{E_+ - E_-}{2c_k} \Delta_k$$

Inilah persis yang ada di kode Anda:

```
g_k = (E_plus - E_minus) / (2 * ck * delta_k)
```

4. Update Parameter (Gradient Descent):

$$\theta_{k+1} = \theta_k - a_k \hat{g}_k$$

Di kode Anda: `theta_k = theta_k - ak * g_k`.

5. Penjadwalan Step Size:

Step size a_k dan perturbasi c_k menurun seiring iterasi:

$$a_k = \frac{a}{(k + A)^\alpha}$$

$$c_k = \frac{c}{k^\gamma}$$

Di kode Anda: `a = 0.1, c = 0.1, alpha = 0.602, gamma_spsa = 0.101, A = 10`.

Mengapa SPSA Efisien?

Setiap iterasi hanya membutuhkan **2 evaluasi** fungsi biaya (E_+ dan E_-), tidak peduli berapa pun dimensi p . Bandingkan dengan finite-difference biasa yang butuh $2p$ evaluasi.

7.2. Mengapa SPSA Sangat Rentan terhadap Barren Plateau?

Sekarang kita kaitkan dengan fenomena Barren Plateau.

Di setiap iterasi SPSA, update parameter adalah:

$$\theta_{k+1} = \theta_k - a_k \hat{g}_k$$

Agar optimasi berjalan efektif, kita membutuhkan dua hal:

1. **Arah gradien yang informatif:** $\hat{g}_k$ harus secara rata-rata menunjuk ke arah penurunan energi yang signifikan.
2. **Magnitudo step yang proporsional:** $a_k |\hat{g}_k|$ harus cukup besar untuk membuat perubahan berarti pada parameter.

Apa yang terjadi di Barren Plateau?

Jika variansi gradien sangat kecil, maka hampir semua komponen $\hat{g}_k$ akan sangat dekat dengan nol:

$$\text{Var}[\hat{g}_k] \approx 0 \quad \Rightarrow \quad |\hat{g}_k| \approx 0$$

Akibatnya:

$$\theta_{k+1} - \theta_k = -a_k \hat{g}_k \approx 0$$

Parameter hampir tidak berubah. Algoritma berjalan di tempat. Bahkan dengan ribuan iterasi, θ_k hanya akan berkelana secara acak di sekitar titik awal, tanpa kemajuan berarti menuju minimum global.

Koneksi dengan Entropi Von Neumann:

- Entropi $S \approx 1 \Rightarrow$ Ansatz menghasilkan state yang hampir teralitkan maksimal secara acak $\Rightarrow$ Variansi gradien $\rightarrow 0 \Rightarrow \hat{g}_k \approx 0 \Rightarrow$ SPSA mandek.
- Inilah mengapa memantau entropi adalah **early warning** bagi kegagalan SPSA.

7.3. Bagaimana Informasi Entropi Membantu Optimasi?

Sekarang kita masuk ke inti **peran entropi Von Neumann** dalam membuat keputusan praktis.

A. Sebelum Optimasi: Memilih Arsitektur Ansatz (Desain Sirkuit)

Ini adalah aplikasi paling kuat. Anda tidak perlu menjalankan VQE penuh untuk mengetahui apakah sebuah ansatz akan bermasalah.

Protokol Diagnostik Pra-Pelatihan:

1. Pilih kandidat depth $\{d_1, d_2, \dots\}$.

2. Untuk setiap depth, bangkitkan M set parameter acak θ (misal $M = 100$).
3. Untuk setiap θ , hitung state $|\psi(\theta)\rangle$ dan entropi Von Neumann $S(\rho_A(\theta))$.
4. Hitung rata-rata entropi: $\bar{S} = \frac{1}{M} \sum S(\theta)$.
5. **Kriteria Pemilihan:**
 - Jika $\bar{S} \approx 1.0$: Ansatz terlalu dalam, kemungkinan besar Barren Plateau. **Tolak.**
 - Jika $\bar{S} \approx 0$: Ansatz terlalu dangkal, tidak cukup ekspresif. **Tolak.**
 - Jika $0.3 < \bar{S} < 0.8$: Ansatz berada di zona aman. **Terima untuk pengujian lebih lanjut.**

Dengan protokol ini, Anda bisa mempersempit kandidat depth dari 12 menjadi hanya beberapa (misal 4, 5, 6) **tanpa menjalankan satu pun iterasi SPSA**. Inilah nilai ekonomis utama dari entropi.

B. Selama Optimasi: Memonitor Kesehatan Pelatihan

Meskipun ansatz sudah dipilih dengan baik, tidak ada jaminan bahwa selama pelatihan sistem tidak akan berkeliaran ke zona berbahaya. Dengan memonitor entropi dan variansi gradien secara real-time (seperti yang Anda lakukan di **halaman 9**):

```
history_grad_var.append(grad_variance)
history_entropy.append(entropy)
```

Anda mendapatkan **dashboard kesehatan**:

- **Entropi stabil di zona sedang, variansi gradien sehat:** Optimasi berjalan normal. Lanjutkan.
- **Entropi tiba-tiba melonjak ke 1.0, variansi gradien drop:** Ada kemungkinan sistem memasuki Barren Plateau. Mungkin perlu **restart** dengan initial parameter baru, atau **mengurangi learning rate**.
- **Entropi sangat rendah, energi masih tinggi:** Ansatz mungkin kurang ekspresif. Tidak banyak yang bisa dilakukan selain menambah depth.

C. Setelah Optimasi: Verifikasi Kualitas Solusi

Entropi solusi akhir juga memberikan wawasan:

- Jika energi minimum dicapai dengan entropi yang sangat rendah, solusi mungkin adalah *product state* yang terlalu sederhana dan tidak menangkap korelasi kuantum yang diperlukan.
- Jika energi minimum dicapai dengan entropi yang moderat, solusi kemungkinan memanfaatkan keteralitan secara optimal.

7.4. Simpulan Akhir: Narasi Lengkap

Mari kita simpulkan seluruh perjalanan dari Tahap 1 hingga 7 dalam satu paragraf naratif:

Dalam VQE untuk optimasi portofolio, kita menggunakan ansatz kuantum yang terdiri dari layer rotasi dan entanglement (Tahap 4) untuk membangun state 2-qubit $|\psi(\theta)\rangle$ (Tahap 1). Keteralitan antara qubit diukur dengan menghitung matriks densitas tereduksi melalui partial trace (Tahap 2) dan kemudian menghitung entropi Von Neumann $S(\rho_A) = -\sum \lambda_i \log_2 \lambda_i$ (Tahap 3). Ketika ansatz terlalu dalam, ia berperilaku seperti desain acak, menyebabkan state terdistribusi seragam, yang mengakibatkan keteralitan maksimal ($S \rightarrow 1$) dan variansi gradien $\rightarrow 0$, sebuah fenomena yang dikenal sebagai Barren Plateau (Tahap 5). Entropi Von Neumann dan variansi gradien adalah dua gejala dari penyebab yang sama, sehingga entropi dapat berfungsi sebagai proksi deteksi dini (Tahap 6). Dalam algoritma SPSA (Tahap 7), informasi ini digunakan untuk memilih depth ansatz yang optimal (sebelum pelatihan), memonitor kesehatan optimasi (selama pelatihan), dan memverifikasi kualitas solusi (setelah pelatihan). Eksperimen Anda menunjukkan bahwa depth 5-6 adalah "sweet spot" di mana ansatz cukup ekspresif (entropi moderat) namun tetap terlatih (variansi gradien sehat), menghasilkan solusi energi rendah yang stabil.

7.5. Pesan Penutup

Anda telah menyelesaikan **Roadmap Belajar 7 Tahap**. Ini adalah perjalanan yang panjang dan mendalam, dari fondasi matematika kuantum paling dasar hingga interpretasi hasil eksperimen nyata.

Beberapa poin penting untuk dibawa pulang:

1. **Entropi Von Neumann adalah "termometer ekspresifitas"** ansatz kuantum Anda.
2. **Keseimbangan adalah kunci:** Terlalu sedikit keteralitan $\Rightarrow$ underfitting. Terlalu banyak $\Rightarrow$ Barren Plateau.
3. **Grafik Anda bukan sekadar plot**, melainkan alat diagnostik yang didukung oleh teori matematika yang ketat.
4. **Pendekatan ini skalabel:** Untuk sistem dengan lebih banyak qubit, menghitung entropi subsistem tunggal (yang selalu matriks 2x2) tetap murah, sementara mendeteksi Barren Plateau secara langsung menjadi jauh lebih mahal.

Jika ada bagian dari roadmap ini yang ingin Anda perdalam, atau jika Anda ingin mendiskusikan modifikasi algoritma, jangan ragu untuk bertanya. Selamat menjelajahi dunia komputasi kuantum lebih dalam!